

자연어 처리 NLP (Natural Language Processing, NLP)

토큰화

- 자연어 : 자연언어, 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어 (<→ 프로그래밍 언어)

자연어 처리

- 인공지능의 하위 분야로, 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
- 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델 개발
- 이러한 모델 개발 위해 해결되어야 할 문제

1. 모호성(Ambiguity)

- 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우 다수
- 이런 다양한 의미를 이해하고 명확하게 구분할 수 있어야 함

2. 가변성(Variability)

- 인간의 언어는 다양한 사투리(Dialects), 강세(Accent), 신조어(Coined word), 작문 스타일로 인해 매우 가변적
- 이러한 가변성을 처리할 수 있어야 함 + 사용 중인 언어를 이해할 수 있어야 함

3. 구조(Structure)

- 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하여 의미(Semantic)를 해석
- 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 함

- 이러한 문제를 이해하고 구분할 수 있는 모델을 만드려면 **말뭉치(Corpus)를 일정한 단위인 토큰(Token)으로 나누어야 함**

◦ 말뭉치

- 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어 → 뉴스 기사, 사용자 리뷰, 저널 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터
- 일련의 단어의 가능성을 예측하는 알고리즘인 언어 모델 구축 평가에 자주 사용됨

◦ 토큰

- 개별 단어나 문장 부호와 같은 텍스트 → 말뭉치보다 더 작은 단위
- 텍스트의 개별 단어, 구두점 또는 기타 의미 단위일 수 있음
- 토큰으로 나누는 목적 : 컴퓨터가 자연어를 이해할 수 있게 나누는 과정 = **토큰화(Tokenization)**

• 토큰화(Tokenization)

- 컴퓨터가 텍스트를 보다 효율적으로 분석하고 처리할 수 있도록 하는 중요 단계
- 토큰화를 위해 **토큰나이저(Tokenizer)** 사용 : 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 SW

• 토큰나이저(Tokenizer)

- 텍스트에서 발생하는 다양한 단어와 구문을 식별하고 분석할 수 있게 함 → 언어 모델링 또는 기계 번역과 같은 다양한 자연어 처리 작업에서 사용됨

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', '느', '수', '있', '다', '.']

• 토큰을 나누는 기준

- 구축하려는 시스템이나 상황에 따라 다름
- 어떻게 토큰을 나누었느냐에 따라 시스템의 성능이나 처리 결과가 크게 달라지기도 함

- **토큰나이저 구축 방법**

1. **공백 분할** : 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
2. **정규표현식 적용** : 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할
3. **어휘 사전 (Vocabulary) 적용** : 사전에 정의된 단어 집합을 토큰으로 사용
 - 사전에 정의된 단어를 활용해 토큰나이저를 구축 → 직접 어휘 사전을 구축하기 때문에 없는 단어나 토큰이 존재할 수 있음 = **OOV(Out of Vocab)** → OOV 문제를 고려하여 토큰화하는 것이 중요
 - 큰 어휘 사전을 구축시 문제 : 학습 비용의 증가, **차원의 저주(Curse of Dimensionality)**에 빠질 수 있음
 - 모든 토큰이나 단어를 벡터화하면 어휘 사전에 등장하는 토큰 만큼의 차원이 필요
 - 벡터값이 거의 모두 0의 값을 가지는 **희소(sparse)** 데이터로 표현됨
 - 희소 데이터의 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려하기 때문에 문장에서 발생한 토큰들의 순서 관계를 잘 표현하지 못함 (ex) I like cats = Cats like I
4. **머신러닝 활용** : 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용

단어 및 글자 토큰화

- 토큰화 : 자연어 처리에서 매우 중요한 전처리 과정 → 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업 → 정확한 분석을 위해 필수, 단어나 문장의 빈도수, 출현 패턴 등을 파악할 수 있음
- 작은 단위로 분해된 텍스트 데이터 : 컴퓨터가 이해하고 처리하기 용이 → 기계 번역, 문서 분류, 감성 분석 등 다양한 자연어 처리 작업에 활용
- 입력된 텍스트 데이터를 단어(Word)나 글자(Character) 단위로 나누는 기법 : 단어 토큰화, 글자 토큰화
 - 이러한 기법을 통해 각각의 토큰은 의미를 갖는 최소 단위로 분해됨
 - 최근 더 정교한 토큰화 기법이 연구되고 있지만, 여전히 단어 토큰화나 글자 토큰화처럼 간단하고 명료한 아이디어가 강력한 토큰화 기법으로 사용되고 있음

1. 단어 토큰화(Word Tokenization)

- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행됨
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용되며 가장 일반적인 토큰화 방법

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

- 문자열 데이터 형태는 split 메서드를 이용 : 주어진 구분자를 통해 문자열을 리스트 데이터로 나누어줌
 - 구분자를 입력하지 않으면 공백(Whitespace)를 기준으로 나눔
- OOV : 'cg.'와 'cg'는 다른 토큰 / '최고!'와 '최고!!'는 다른 토큰 → **한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약**

2. 글자 토큰화(Character Tokenization)

- 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식 → **[장점] 비교적 작은 단어 사전을 구축할 수 있음**
 - 작은 단어 사전을 사용하면 학습 시 컴퓨터 자원 절약 + 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용 → (ex) 다음 문자를 예측하는 언어 모델링

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!', ' ']
```

- 글자 토큰화는 리스트 형태로 변환하면 쉽게 수행 가능
- 공백도 토큰으로 나뉘, 영어는 각 알파벳으로 나뉘
- 한국어의 한 글자는 여러 자음과 모음의 조합으로 이루어짐 → 자로 단위로 나누어 자소 단위 토큰화 수행

자모 라이브러리 활용

- 한글 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리 → 텍스트를 자소 단위로 분해해 토큰화 수행

1. 자모 변환 함수

자모 변환 함수

```
retval = jamo.h2j(  
    hangul_string  
)
```

- 입력된 한글 문자열을 유니코드 U+1100 ~ U+11FE 사이의 조합형 한글 자모로 변환하는 함수
- (cf) 컴퓨터가 한글을 인코딩하는 방식 : 조합형, 완성형
 - 조합형 : 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현
 - 완성형 : 조합된 글자 자체에 값을 부여해 인코딩하는 방식
- h2j 함수 : 완성형으로 입력된 한글을 조합형 한글로 변환

```
from jamo import h2j  
print(h2j("강"))
```

```
# 강 (U+1100, U+1161, U+11BC)  
# → 조합형 자모: 초성(ㄱ), 중성( ㅏ), 종성(ㅇ)
```

2. 한글 호환성 자모 변환 함수

한글 호환성 자모 변환 함수

```
retval = jamo.j2hcj(  
    jamo  
)
```

- j2hcj 함수 : 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수
- 조합형 한글로 입력된 문자열은 초성, 중성, 종성으로 나뉘 → 이 함수를 통해 완성형 한글을 쉽게 자소 단위로 나눌 수 있음

```
from jamo import h2j, j2hcj  
print(j2hcj(h2j("강")))
```

```
# ㄱ ㅏ ㅇ (호환형 자모)  
# → 초성: ㄱ / 중성: ㅏ / 종성: ㅇ
```

- 자소단위 토큰화

예제 5.3 자소 단위 토큰화

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

출력 결과

```
['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄴ', ' ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', ' ', 'ㄷ', 'ㄱ', 'ㅣ', ' ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅏ', 'ㄱ', 'ㅈ', 'ㅈ', 'ㄱ', ' ', 'ㅅ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅈ', 'ㅈ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ', ' ', 'ㅈ', 'ㅏ', 'ㅍ', 'ㄱ', 'ㅏ', 'ㅣ', ' ', 'ㄷ', 'ㅈ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㅇ', 'ㅈ', ' ', 'ㅇ', 'ㅅ', 'ㅌ', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅈ', 'ㄷ', 'ㅈ', 'ㅇ', 'ㅌ', 'ㄱ', ' ', 'ㅈ', 'ㅏ', 'ㄱ', 'ㅏ', 'ㅣ', 'ㅣ']
```

- [장점]
 - 글자 단위로 토큰화 시 (단어 단위로 토큰화하는 것에 비해) **비교적 적은 크기의 단어 사전 구축 가능**
 - **작은 크기의 단어 사전으로도 OOV를 획기적으로 줄일 수 있음**
- [단점]
 - 개별 토큰은 의미가 없음 → 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야함
 - 토큰 조합 방식을 사용해 문장 생성이나 개체명 인식(Name Entity Recognition)등을 구현할 경우, 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있음
 - 모델 입력 시퀀스(sequence)의 길이가 길어질수록 연산량이 증가
 - (ex) 단어 토큰화의 경우 입력 시퀀스의 길이가 13, 클자 토큰화의 경우 53, 자소 단위 토큰화의 경우 101까지 늘어남

형태소 토큰화(Morpheme Tokenization)

- 텍스트를 형태소 단위로 나누는 토큰화 방법 → 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미있는 단위로 분류하는 작업
- 한국어와 같이 교착어(Agglutinative language)인 언어에서 중요하게 수행됨 → 각 단어가 띄어쓰기로 구분되지 않음 → 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이룸 → 각 형태소를 적절히 구분해 처리 필요
- 형태소(실제로 의미를 가지고 있는 최소의 단위)
 - 자립 형태소: 스스로 의미를 가짐 → 명사, 동사, 형용사
 - 의존 형태소: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용 → 조사, 어미, 접두사, 접미사

형태소 어휘 사전(Morpheme Vocabulary)

- 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전
- 단어가 어떤 형태소들의 조합으로 어우러져 있는지에 대한 정보를 담고 있음 → 형태소 분석 작업에서 중요한 역할

“그는 나에게 인사를 했다.”
“나는 그에게 인사를 했다.”

- 형태소 어휘 사전은 이 문장들을 다음과 같이 형태소 단위로 분해해 저장
 - ‘그는’, ‘나에게’, ‘인사를’, ‘했다’, ‘나는’, ‘그에게’
- 형태소 어휘 사전이 필요한 이유
 - 만약 “그녀는 그에게 인사를 했다”라는 문장이 새로 등장한다면, 모델이 ‘그는’, ‘나는’, ‘그녀는’을 같은 의미 단위로 인식하지 못하고 학습을 어렵게 함
 - 이를 방지하기 위해, 형태소 단위로 사전을 구성한다면
 - [‘ㄴ은’, ‘-에게’, ‘그’, ‘나’ 등] 정보를 바탕으로 → ‘그녀의’ 토큰 정보도 쉽게 확장 가능해짐
 - ‘그녀는’, ‘그녀에게’ 같은 어휘도 빠르게 학습 가능
- 형태소 어휘 사전에 포함되는 정보

- 각 형태소가 어떤 품사(Part of Speech, POS)에 속하는지
- 해당 품사의 뜻 등

- **품사 태깅 (POS Tagging)**

- 형태소 분석 후 각 형태소에 해당하는 **품사**를 태깅하는 작업 → 문맥 고려가 가능하고, 자연어 처리의 정확도가 향상

“그는 나에게 인사를 했다” → 품사 태깅 결과:

- 그(명사)
- 는(조사)
- 나(명사)
- 에게(조사)
- 인사(명사)
- 를(조사)
- 했다(동사)

KoNLPy 라이브러리

- Okt 형태소 분석기
- 꼬꼬마 형태소 분석기

NLTK 라이브러리

- Punkt 모델
- Averaged Perceptron Tagger 모델

spaCy 라이브러리

하위 단어 토큰화(Subword Tokenization)

기존 형태소 분석기의 문제점

- 형태소 분석은 자연어 처리의 중요한 전처리 과정이며, 단어를 최소 의미 단위로 나눔
- 그러나 현대어에는 다음과 같은 문제가 많아 형태소 기반 분석이 어려움:
 - 신조어, 외래어, 오타자, 띄어쓰기 오류
 - 긴 복합어, 전문용어 등
- 예시 1
 - 문장: “시보리도 짹짹하고 허리도 어빙하지안구 죠하효”
 - 형태소 분석 결과: [시, 보리, 도, 짹짹하고, 허리, 도, 어, 정하지안구, 죠, 하, 효]
 - ‘시보리도’를 ‘시’와 ‘보리’로 잘못 분석
- 예시 2
 - 문장: “진짜 멋있네요! 돈썰날만 하네요.”
 - 결과: [진짜, 멋있네요, !, 돈썰날, 만, 하네요, .]
 - ‘돈썰날만’에서 ‘돈썰’, ‘날’, ‘만’ 혹은 ‘돈썰’, ‘날만’ 으로 분리될 수도 있음

하위 단어 토큰화의 기본 개념

- 하나의 단어를 더 작은 하위 단어 단위(Subword)로 쪼개어 토큰화하는 방법 → OOV 문제, 신조어, 오류 등에도 강인함
- (ex) 단어: Reinforcement → 하위 단어로 분해: Rein , force , ment
- 장점
 - 긴 단어도 잘게 나눠 처리 가능 → **속도 향상**
 - **OOV 단어 대응** 가능

- 신조어, 오타자, 고유명사 등에도 유연하게 대처

바이트 페어 인코딩 방법과 예시

- 센텐스피스와 코포라 라이브러리
- 토큰나이저 모델 학습 과정

워드피스 방법과 예시

- 토큰나이저스 라이브러리
- 토큰나이저 모델 학습 과정

임베딩

- 토큰화만으로 모델을 학습하기에 한계 → 컴퓨터는 텍스트 자체를 이해할 수 없음
- **텍스트 벡터화(Text Vectorization)** 과정 필요 : 텍스트를 숫자로 변환하는 과정

1. One-Hot Encoding(원-핫 인코딩)

- 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑 → 해당 색인 위치를 1로 표시 나머지 위치는 모두 0으로 표시하는 방식

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

2. Count Vectorization(빈도 벡터화)

- 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식
- (ex) apples 단어 총 4번 등장 → 벡터값 4

- **단어나 문장을 벡터 형태로 변환하기 쉽고 간단하다는 장점이 있지만, 벡터의 희소성(Sparsity)이 크다는 단점이 존재**

• 희소성(Sparsity)의 문제점

- 말뭉치 내 존재하는 모든 토큰의 개수만큼 벡터 차원을 가져야 함 → 그러나 입력 문장에 등장하는 토큰 수는 그보다 훨씬 적기 때문에:
 - 컴퓨팅 비용 증가
 - 차원의 저주 문제 발생
- 또한, 텍스트 벡터가 문장의 의미를 내포하지 않기 때문에 → 두 문장이 의미적으로 유사해도 벡터는 전혀 다르게 나타날 수 있음
 - 의미를 반영하지 못하는 예시

i scream for ice cream
ice cream for i scream

- 위 두 문장에 원-핫 인코딩이나 빈도 벡터화를 적용하면:

→ [1, 1, 1, 1] 과 같이 동일한 벡터로 표현됨

→ 하지만 두 문장은 의미가 다르기 때문에 이 표현 방식은 문제를 가짐

• 해결 방법

◦ 워드 임베딩 (Word Embedding)

- 의미 문제를 해결하기 위해 사용되는 기법: Word2Vec, FastText
- [장점]

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법 → 단어 의미를 벡터 공간에 투영 → 다른 단어들과의 상대적 위치로 단어 간 관계 추론 가능
- [단점]
 - 워드 임베딩은 **고정된 임베딩만 학습**하기 때문에 → 다의어, 다양한 문맥 정보를 반영하는 데 한계가 있음
 - 해결방안 : 인공 신경망을 활용해 **동적 임베딩 (Dynamic Embedding)** 기법 사용

언어 모델

- **언어 모델(Language Model)**: 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
→ 이를 위해 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행
- 언어 모델은 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용

P(안녕하세요 반갑습니다.) = 0.081		P(서울특별시) = 0.59	
Input : 안녕하세요	...	Input : 대한민국 ... 강남구	...
P(아니오, 괜찮습니다.) = 0.000001		P(아이스 아메리카노) = 0.000001	

그림 6.1 언어 모델

- ‘안녕하세요’라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어려움 → 이는 비지도 학습 문제로 모델 스스로 문장 등장 확률을 계산해야 함
- 또한, ‘안녕하세요’ 다음에 올 수 있는 문장은 사실상 무한에 가까움 → 그러므로 완성된 문장 단위로 확률을 계산하는 것은 불가능
- 주어진 문장 뒤에 나올 수 있는 문장은 매우 다양하기 때문에 → 완성된 문장 단위로 확률을 계산하는 것은 어려운 일
- 이러한 문제를 해결하기 위해 문장 전체를 예측하는 방법 대신에 → 하나의 토큰 단위로 예측하는 방법인 자기회귀 언어 모델이 고안

1. 자기회귀 언어 모델(Autoregressive Language Model)

- **자기회귀 언어 모델(Autoregressive Language Model)**: 입력된 문장들의 **조건부 확률**을 이용해 다음에 올 단어를 예측
- 언어 모델에서 조건부 확률 : 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것
 - 이를 위해 이전에 등장한 모든 토큰의 정보를 고려하여 → 문장의 문맥 정보를 파악하고 다음 단어를 생성
 - 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, → 이 과정이 반복

수식 6.1 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

조건부 확률의 연쇄법칙

- 언어 모델에서 조건부 확률을 계산하기 위해 → 이전에 등장한 단어 시퀀스($w_1, w_2, \dots, w_{i-1}$)를 기반으로 다음 단어 w_i 의 확률을 계산
- 수식 6.1에 조건부 확률의 연쇄법칙(Chain Rule for conditional probability)을 적용

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

- 언어 모델에서 조건부 확률은 연쇄법칙을 이용해 계산됨
 - 이때 하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률로 계산하는 방식
 - 이전 단어들의 시퀀스가 주어졌을 때 다음 단어의 확률을 이전 단어들의 조건부 확률을 이용해 계산
 - 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타냄
 - 이를 통해 언어 모델은 문장 전체의 확률을 계산하고 → 그것을 이용해 다음 단어를 예측

- 앞선 '안녕하세요' 다음에 등장할 수 있는 문장을 조건부 확률의 연쇄법칙 방법으로 시각화

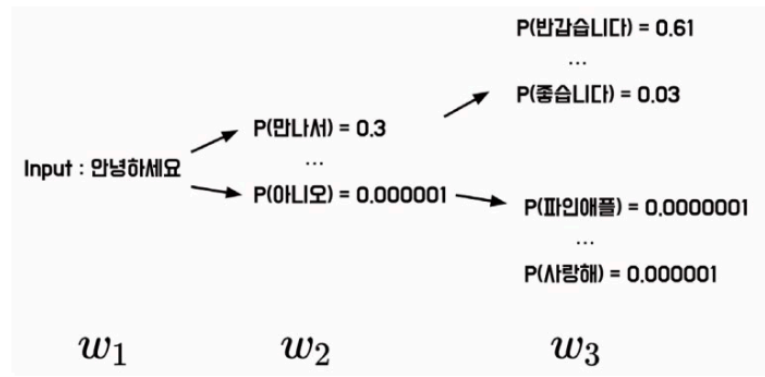


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

예측 순서

- 자기회귀 언어 모델에서는 → 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
- 이를 위해 입력 토큰 w_1 을 바탕으로 → 모델은 토큰 w_2 가 등장할 확률 $P(w_2 | w_1)$ 을 예측
- 그다음 모델의 출력값 w_2 가 다시 입력값이 되어 → 모델은 확률 $P(w_3 | w_1, w_2)$ 를 예측

'자기회귀(AutoRegressive)'라는 이름의 의미

- 이러한 방식으로 **모델의 출력값이 모델의 입력값으로 사용**되기 때문에 → 자기회귀(AutoRegressive)
- 자기회귀 언어 모델은 **시점별로 다음에 올 토큰을 예측하는 것** → **토큰 분류 문제**로 정의 가능

2. 통계적 언어 모델(Statistical Language Model)

- 통계적 언어 모델(Statistical Language Model)
 - 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
 - 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악하고 다음에 등장할 단어의 확률을 예측
- 일반적으로 통계적 언어 모델은 마르코프 체인(Markov Chain)을 이용해 구현
 - 마르코프 체인은 **빈도 기반의 조건부 확률 모델** 중 하나 → 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측
- 빈도 기반의 조건부 확률 모델에서는 주어진 데이터에서 **각 변수가 발생한 빈도수**를 기반으로 확률을 계산

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- '안녕하세요'라는 문장이 1,000번 등장
 - 이어서 '안녕하세요 만나서'가 700번
 - '안녕하세요 반갑습니다'가 100번 등장
- 빈도 기반의 조건부 확률 모델 단점
 1. 단어의 순서와 빈도에만 기초해 문장의 확률을 예측
 - 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성 가능성
 2. 또한, 빈도 등장량이 적은 단어나 문장에 대해서는
 - 정확한 확률을 예측하기 어려운 한계

- 통계적 언어 모델 장점
 - 하지만 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 **패턴을 찾아 확률 분포를 생성**
 - 이를 이용해 새로운 문장을 생성 가능
 - 다양한 종류의 텍스트 데이터를 학습 가능
 - 통계적 언어 모델은 **대규모 자연어 데이터를 처리하는 데 효과적**
 - 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현 가능

사전학습 기반 언어 모델: GPT, BERT

- 최근 연구들은 자연어 처리 기법을 **언어 모델을 활용해 가중치를 사전 학습**
- 예를 들어 트랜스포머 모델에 기반한 언어 모델인
 - GPT(Generative Pre-trained Transformer)
 - BERT(Bidirectional Encoder Representations from Transformers)

→ 다음과 같은 **문장 생성 기반**을 이용해 모델을 사전 학습

1. GPT

- **Raw Sentence:** GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
 - Input: GPT는 OpenAI에서 개발한
 - Prediction-1: 인과적
 - Prediction-2: 언어 모델입니다.

2. BERT

- **Raw Sentence:** BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
 - Input: BERT는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
 - Prediction: Google, 마스킹된

- **GPT와 BERT 모델은 자연어 처리 분야에서 혁신적인 성과를 거둔 언어 모델 중 하나**
- GPT와 BERT 모델과 같이 신경망을 통해 확률 P를 계산할 수도 있지만, → 통계적인 방법을 통해 확률을 계산할 수도 있음 (7장)

N-gram

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 → 특정 단어 시퀀스가 등장할 확률을 추정

N-gram 모델 개념 및 명칭

- N-gram 모델은 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 → **N개의 토큰을 묶어서 분석**
- 이때, 연속된 N개의 단어를 하나의 단위로 취급하여 추론하는 모델이며
 - N이 1일 때는 **유니그램(Unigram)**
 - N이 2일 때는 **바이그램(Bigram)**
 - N이 3일 때는 **트라이그램(Trigram)**
 - N이 4 이상이면 일반적으로 **N-gram**

Unigram : <u>안녕하세요</u> 만나서 <u>진심으로</u> 반가워요	안녕하세요. 만나서. <u>진심으로</u> . 반가워요
Bigram : <u>안녕하세요</u> 만나서 <u>진심으로</u> 반가워요	안녕하세요 만나서. 만나서 <u>진심으로</u> . <u>진심으로</u> 반가워요
Trigram : <u>안녕하세요</u> 만나서 <u>진심으로</u> 반가워요	안녕하세요 만나서 <u>진심으로</u> . 만나서 <u>진심으로</u> 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

N-gram의 조건부 확률

- N-gram 언어 모델은 모든 토큰을 사용하지 않고 → **N-1개의 토큰만을 고려해 확률을 계산**
- 따라서, 각 N-gram 언어 모델에서 t번째 토큰의 조건부 확률

$$P(w_t \mid w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- w_t : 예측하려는 단어
- $w_{t-1}, \dots, w_{t-N+1}$: 예측에 사용되는 이전 단어들 → N의 크기를 조정하면 성능도 조정 가능

모델 실습

```
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

# NLTK 버전
unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

출력 결과

```
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

1. 통계적 언어 모델은 **상대적으로 구현이 쉬움** → 간단한 파이썬 코드 또는 NLTK 라이브러리로 구현 가능
2. **N-gram은 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석하는 데 큰 효과** → 관용 표현 분석에도 활용
 - "밥이 무겁다"라는 표현은 "비밀을 잘 지킨다"는 뜻
 - 따라서 자주 등장하는 연속된 단어 구를 추출 → **관용적 표현의 의미를 파악 가능**
3. 자연어 처리에서의 응용
 - **N-gram은 시퀀스에서 연속된 N개의 단어를 추출** → 단어의 순서가 중요한 자연어 처리 작업 및 문서 패턴 분석에 활용

TF-IDF

- **TF-IDF (Term Frequency-Inverse Document Frequency)**
 - 텍스트 문서에서 특정 단어의 **중요도**를 계산하는 방법
 - 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미
- 즉, TF-IDF는 BoW(Bag-of-Words)에 가중치를 부여하는 방법

BoW 개념 정리

- **BoW** : 문서나 문장을 단어의 집합으로 표현하는 방법 → 문서나 문장에 등장하는 단어의 중복을 허용해 **빈도를 기록**
- 원-핫 인코딩은 단어의 등장 여부만 판단해 0과 1로 표현 ↔ **BoW는 등장 빈도까지 고려해 표현**

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW는 모든 단어에 **동일한 가중치**를 부여 → 예: 영화 리뷰의 긍정/부정 분류에 활용할 경우 높은 성능을 얻기 어려움
 - "I like this movie" 와 "I don't like this movie"는 'don't'가 핵심이지만
 - 'don't'가 자주 등장하지 않으면 토큰라이저가 **무시할** 가능성

1. 단어 빈도 (TF: Term Frequency)

- 단어 빈도 : **문서 내에서 특정 단어의 등장 횟수**를 나타내는 값
 - 3개의 문서에서 **movie** 라는 단어가 4번 등장하면 → 해당 단어의 TF 값은 **4**

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

- TF는 BoW 벡터처럼 문서 내 단어 빈도수를 계산 → **해당 단어의 상대적인 중요도**를 측정하는 데 사용
 - 단어가 문서 내에 자주 등장하면 **TF 값이 커지며** → 해당 단어는 중요한 역할을 할 수 있음
 - 그러나 TF는 문서 길이가 길어질수록 해당 단어의 TF 값도 **함께 높아질 수 있다는 한계**

2. 문서 빈도 (DF: Document Frequency)

- 문서 빈도 : **한 단어가 얼마나 많은 문서에 등장했는지를 의미** → 여러 문서에서 등장한 횟수를 기준으로 DF를 측정
 - 3개의 문서에서 **movie** 가 4번 등장했더라도 → 3개 문서 모두에서 등장했다면 DF는 **3**

수식 6.6 문서 빈도

$$DF(t, D) = count(t \in d : d \in D)$$

- DF 값이 높을수록 → 그 단어는 일반적이며 중요도가 낮을 수 있음
- DF 값이 낮을수록 → 특정한 문맥에서만 사용되는 **의미 있는 단어일** 가능성이 높음

3. 역문서 빈도 (IDF: Inverse Document Frequency)

- 역문서 빈도(IDF) : 전체 문서 수를 DF로 나눈 다음 로그를 취한 값 → 문서 내 **특정 단어의 중요도를 평가**
 - 문서 빈도가 높을 수록 : 해당 단어가 일반적이고 상대적으로 중요하지 않다는 의미
 - 문서 빈도의 역수를 취하면 : 단어의 빈도수가 적을 수록 IDF 값이 커지게 보정하는 역할

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- DF가 0이면 분모가 0이 되어버리므로 → **분모에 1을 더해 안정화**
- 문서 수가 많은 상황에서는 IDF 값이 너무 커질 수 있으므로 → **로그를 취해 조정**
 - (ex) 10000개의 문서에서 특정한 단어가 1번만 등장한다면 IDF 값은 5000이 됨

4. TF-IDF 최종 수식

- **TF-IDF** : TF와 IDF의 곱

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 해당 단어가 자주 등장하면서 → 전체 문서에서는 드물게 등장하는 단어일수록 **TF-IDF 값이 커짐**
 - 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아짐

모델 실습

Scikit-learn 라이브러리 설치

```
pip install scikit-learn
```

- 사이킷런 라이브러리는 **말뭉치를 쉽게 TF-IDF 형태로 변환**할 수 있게 → `TfidfVectorizer` 클래스를 제공

TF-IDF 클래스 예시

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(
    input="content",
    encoding="utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1, 1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
    smooth_idf=True,
)
```

- 입력값 `input` : 입력된 데이터의 형태
 - 기본값 `"content"` 는 문자열 또는 바이트형 입력
 - 파일을 사용할 경우 `filename` 으로 입력
- 인코딩 `encoding` : 바이트 또는 파일 입력 시 사용할 텍스트 인코딩 방식 (예: `"utf-8"`)
- 소문자 변환 `lowercase` : 모든 입력 텍스트를 소문자로 변환 여부 (`True` 면 소문자로 변환)
- 불용어 `stop_words` : 불용어 처리 여부. 의미 없는 단어를 제거함.
- N-gram 범위 `ngram_range=(1, 1)` : 사용할 **N-gram의 범위** 설정
 - `(1, 1)` 은 unigram만, `(1, 2)` 는 unigram + bigram까지 포함
- 최대값 문서 빈도 `max_df` : 전체 문서 중 일정 비율 이상 등장한 단어는 **무시** (너무 흔한 단어 제거)
- 최소값 문서 빈도 `min_df` : 전체 문서 중 일정 횟수 **미만으로 등장한 단어**는 무시

- 단어사전 `vocabulary` : 사용자 정의 단어 사전 사용 가능. 없으면 자동 생성
- IDF 분모처리 `smooth_idf=True` : 수식 6.7에서 **분모에 1을 더해** 안정화

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

출력 결과

```
[[0.         0.         0.39687454 0.39687454 0.         0.79374908 0.2344005 ]
 [0.61980538 0.         0.         0.         0.61980538 0.         0.48133417]
 [0.4804584  0.63174505 0.         0.         0.4804584  0.         0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```

- `fit()` : TF-IDF 벡터라이저 학습 (말뭉치 내 단어 빈도 파악)
- `transform()` : 학습된 벡터라이저로 **데이터 변환**
- `toarray()` : 벡터를 넘파이 배열로 변환 → `(문서 수) × (단어 수)` 형태
- `vocabulary_` : 사용된 단어 사전 → 딕셔너리의 키는 단어, 값은 단어의 **색인 인덱스**

- 중요 단어 추출 활용
 - 예: 점수가 가장 높은 3개를 추출하면 → `[[2, 3, 5], [0, 4, 6], [0, 1, 4]]`
 - 단어 사전 기준:
 - `[famous, is, movie]`
 - `[actor, like, that]`
 - `[actor, don, like]`
 - 이를 통해 문서마다 **핵심 단어**를 추출할 수 있음

- TF-IDF의 한계
 - 빈도 기반 벡터화는 문장 순서나 문맥을 고려하지 않는다
 - 예:
 - '나는 바나나를 먹었다'
 - '그녀가 과일을 섭취한다'
 - '고양이는 야옹 운다'
 - 단어의 의미와 문장 유사도는 반영하지 못함
 - 결국, TF-IDF는 단어의 중요도는 반영할 수 있지만
 - **문장의 의미를 벡터에 담기는 어려움**